The polynomial Taylor tree in d dimensions

Claude, with Daniel Murfet

2026-09-06

Abstract

For a polynomial perturbation $J(u) = \sum_{\gamma \in S} c_\gamma u^\gamma$ the multinomial theorem expands $(J(u))^p$ into monomials $u^{\sum_\gamma \kappa_\gamma \gamma}$ indexed by compositions $\kappa \in$ `piAntidiag` $S\, p$, so each layer of the exponential tree (unit 62) is a finite sum of tree terms (unit 60) and the chart standard integral with $\xi = a + J$ is the exact double series $\sum_p \frac{\beta^p}{p!} \sum_\kappa \binom{p}{\kappa} \prod_\gamma c_\gamma^{\kappa_\gamma} \cdot \texttt{treeTerm}(\sum_\gamma \kappa_\gamma \gamma, p)$: the paper's `eq:flucttreeterms` grouped by composition. Zero `sorry`/`axiom`.

1 Setting

Mathlib's `Finset.sum_pow_eq_sum_piAntidiag` is the multinomial theorem over a finite index set; the only bookkeeping is the identification $\prod_\gamma (\prod_i u_i^{\gamma_i})^{\kappa_\gamma} = \prod_i u_i^{\sum_\gamma \kappa_\gamma \gamma_i}$, which is `Finset.prod_comm` plus `Finset.prod_pow_eq_pow_sum`.

2 The things formalised

```
noncomputable def polyJ {d :  } (S : Finset (Fin d →  )) (coef : (Fin d →  ) →  )
    (u : Fin d →  ) :   :=      S, coef   *   i, u i ^   i
def compShift {d :  } (S : Finset (Fin d →  )) (  : (Fin d →  ) →  ) (i : Fin d) :   :=
      S,    *   i
theorem polyJ_pow ... : polyJ S coef u ^ p =      Finset.piAntidiag S p,
      (Nat.multinomial S   :  ) * (     S, coef   ^    ) *   i, u i ^ compShift S   i
theorem treeLayer_polynomial ... : treeLayer   a b c k h (polyJ S coef) p
      =   ^ p / p.factorial *      Finset.piAntidiag S p,
          (Nat.multinomial S   :  ) * (     S, coef   ^    )
            * treeTerm   a b c k h (compShift S  ) p
theorem boxIntegralXi_polynomial_hasSum ... :
    HasSum (fun p =>   ^ p / p.factorial *      Finset.piAntidiag S p, ...)
      (boxIntegralXi   a b c k h (polyJ S coef))
```

3 Proof strategy

The pointwise expansion is the multinomial theorem followed by `prod_mul_distrib`, `prod_comm`, `pow_mul` and `prod_pow_eq_pow_sum`. The layer identity distributes the finite sum through the integral (`integral_finsetSum`, integrability from continuity on the compact box) and pulls out the constants; the series statement is `HasSum.congr_fun` of unit 62.

4 Roadblocks and resolutions

- `Nat.multinomial` and the multinomial theorem live in `Mathlib.Data.Nat.Choose.Multinomial`, outside the import closure.
- `rw [Finset.mul_sum, Finset.sum_mul]` fails when the sum is nested under two products; `simp only` with both lemmas distributes everything.
- `integral_finsetSum` inferred its summand from the integrability proof in Pi form; giving `(f := …)` explicitly as a lambda makes the rewrite match.

5 What was Mathlib, what was new

Mathlib: `Finset.sum_pow_eq_sum_piAntidiag`, `Nat.multinomial`, `Finset.piAntidiag`, `Finset.prod_comm`, `Finset.prod_pow_eq_pow_sum`, `integral_finsetSum`. New: the polynomial tree expansion.

6 Lessons

The paper's grouping of the p-fold products of Taylor coefficients by total multi-index is exactly a sum over compositions in Mathlib's `piAntidiag`; formalising along that grouping avoided any multiset combinatorics.

7 Outcome

For polynomial ξ the full Taylor tree of `thm:TaylorTree` is an exact, convergent double series of tree terms, each of which has leading exponent in $\Lambda(h, k)$ and logarithmic degree at most $d - 1$. The analytic remainder of the theorem is the resummation of this series.